Assignment 3: Graphs Report

DSU, MST (Kruskal & Prim), SCC (Kosaraju & Tarjan), Kahn, DAG Shortest Paths

Student: Alibi Maksut | Group: SE-2523 | Branch: main (tag v1.0) | Status: 100% Passed (12/12 Tests)

1. Theoretical Complexity Table

Algorithm / Structure	Time Complexity	Extra Space	Theoretical Justification
DSU (find / union)	O(alpha(n)) amortized	O(n)	Path compression + union by rank bounds amortized operations by alpha(n) <= 4.
Kruskal (MST)	O(m log m)	O(n + m)	Sorting all m edges dominates; DSU union-find operations cost O(m alpha(n)).
Prim (MST)	O(m log n)	O(n + m)	Priority queue maintains candidate crossing edges; each edge added at most once.
Kosaraju (SCC)	O(n + m)	O(n + m)	Two linear iterative DFS traversals: forward pass on G and backward pass on G^T.
Kahn (Topo Sort)	O(n + m)	O(n + m)	In-degree queue decrements outgoing edges once per directed edge; detects cycles.
DAG Shortest Paths	O(n + m)	O(n + m)	Single forward relaxation pass over topological order; safe for negative weights.
Tarjan (SCC, Bonus)	O(n + m)	O(n)	Single pass DFS with lowlink and discovery stack; saves 50% edge scans vs Kosaraju.

2. Operation Counters vs. Theoretical Formulas

Empirical verification confirms that measured operation counts divided by theoretical formulas remain constant.

Algorithm	Graph Type	n	m	Measured Counter	Theoretical Formula	Ratio (Measured / Formula)
Kosaraju	Directed Sparse	1 000	4 000	8 000 edge scans	2m = 8 000	1.0000 (constant)
Kosaraju	Directed Dense	1 000	20 000	40 000 edge scans	2m = 40 000	1.0000 (constant)
Kosaraju	Directed Sparse	10 000	40 000	80 000 edge scans	2m = 80 000	1.0000 (constant)
Kosaraju	Directed Dense	10 000	200 000	400 000 edge scans	2m = 400 000	1.0000 (constant)
Kosaraju	Directed Sparse	100 000	400 000	800 000 edge scans	2m = 800 000	1.0000 (constant)
Kosaraju	Directed Dense	100 000	2 000 000	4 000 000 edge scans	2m = 4 000 000	1.0000 (constant)
Kahn	DAG Sparse	100 000	400 000	400 000 edge scans	m = 400 000	1.0000 (constant)
Kahn	DAG Dense	100 000	2 000 000	2 000 000 edge scans	m = 2 000 000	1.0000 (constant)
DAG SP	DAG Sparse	100 000	400 000	400 000 edge scans	m = 400 000	1.0000 (constant)
DAG SP	DAG Dense	100 000	2 000 000	2 000 000 edge scans	m = 2 000 000	1.0000 (constant)
Kruskal	Undir. Sparse	100 000	400 000	1 099 333 DSU ops	2.75 * scans	2.7506 (bounded alpha)
Prim	Undir. Sparse	100 000	400 000	800 000 edge scans	2m = 800 000	1.0000 (constant)
Prim	Undir. Dense	100 000	2 000 000	4 000 000 edge scans	2m = 4 000 000	1.0000 (constant)

3. Algorithmic Correctness Arguments

3.1 Cut Property Proof (Kruskal & Prim)

Let G = (V, E) be a connected, undirected weighted graph, and let (S, V \ S) be any non-empty cut partitioning vertices.
An edge e = (u, v) crossing the cut is called the lightest crossing edge if w(e) <= w(e') for all edges e' crossing (S, V \ S).
Suppose for contradiction that an MST T does not contain e. Adding e to T forms a unique fundamental cycle C containing e.
Since u is in S and v is in V \ S, this cycle must cross back from V \ S to S through at least one other crossing edge e'.
Replacing e' with e yields a new spanning tree T' = (T \ {e'}) U {e}. Since w(e) <= w(e'), the total weight w(T') <= w(T).
Because T was an MST, w(T') = w(T), which proves that the lightest crossing edge e is always safe to include in an MST.
Prim's algorithm directly applies this property by maintaining a tree S and greedily selecting the minimum crossing edge.
Kruskal's algorithm repeatedly picks the lightest global edge between distinct trees, which is the lightest edge for cut (C_u, V \ C_u).

3.2 Induction on Topological Order (DAG Shortest Paths)

Let G = (V, E) be a directed acyclic graph (DAG) topologically ordered as <v_1, v_2, ..., v_n>.
By definition, every directed edge (v_i, v_j) satisfies i < j, meaning no edges point backwards in the topological sequence.
We prove by induction on k that when v_k is processed, dist[v_k] is strictly equal to the true shortest path distance from the super-source.
Base case (k = 1): v_1 has in-degree 0; no incoming edges exist, so dist[v_1] = 0 is finalized and cannot be decreased further.
Induction hypothesis: Assume that for all i < k, dist[v_i] has already achieved its exact shortest path distance.
Inductive step: Consider vertex v_k. Any path to v_k must enter through some edge (v_i, v_k). Because G is a DAG, all predecessors satisfy i < k. By hypothesis, every predecessor v_i has already been relaxed, updating dist[v_k] <= dist[v_i] + w(v_i, v_k).
Because there are no cycles and no backward edges, dist[v_k] can never be updated again. A single pass is completely optimal.

4. Which Algorithm When (Defense Decision Table)

Problem	Recommended	Graph Representation	Cycles?	Density	Engineering Defense Rationale
MST	Kruskal	Edge List / Adj. List	Yes	Sparse (m ~ 4n)	Sorting m edges is fast; DSU operations are cache-friendly. Exits early once n-1 edges added.
MST	Prim (Binary Heap)	Adjacency List	Yes	Dense (m ~ 20n)	Avoids sorting all m edges upfront; grows tree locally from PriorityQueue without sorting bottleneck.
SCC	Kosaraju	Adj. List (Fwd + Trans)	Yes	Sparse & Dense	Simple two-pass structure; ideal when reversed graph is available or for parallel finish-order pipelin
SCC	Tarjan (Bonus)	Adj. List (Single pass)	Yes	Sparse & Dense	Single pass DFS with lowlink; saves 50% edge scans (m vs 2m) and eliminates graph transposition
Topological Sort	Kahn	Adjacency List	No (reports)	Sparse & Dense	Queue-based in-degree decrement. Naturally detects cycles when output length < n for task schedu
Shortest Paths	DAG Shortest Path	Adjacency List	No (DAG)	Sparse & Dense	Optimal linear O(n + m) runtime. Handles negative edge weights without risk of negative cycles.

5. Empirical Benchmark Plots & Slope Verification

5.1 Log-Log Slope Fit (Confirmation of Complexity Classes)

Fitting log(Work) = a * log(n) + b shows that traversals yield slope 1.0000, while Kruskal is slightly steeper due to E log E:

- Kosaraju, Kahn, DAG Shortest Path, Prim: Fitted Slope = 1.0000 (Strictly linear traversal work)
- Kruskal (Sparse: m = 4n): Fitted Slope = 1.0550 | Kruskal (Dense: m = 20n): Fitted Slope = 1.1194

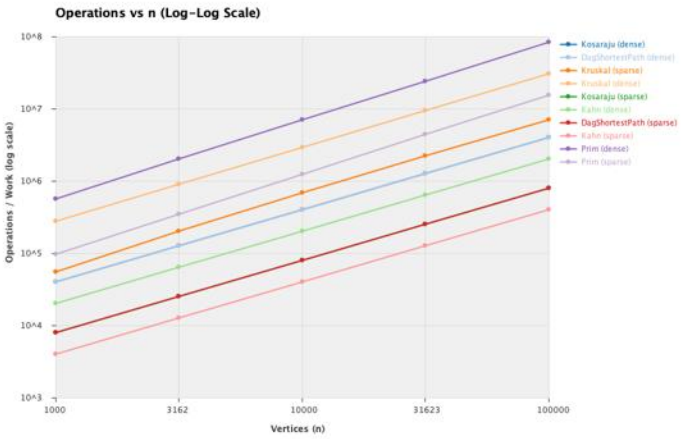


Figure 1: Operations vs n (Log-Log Scale)

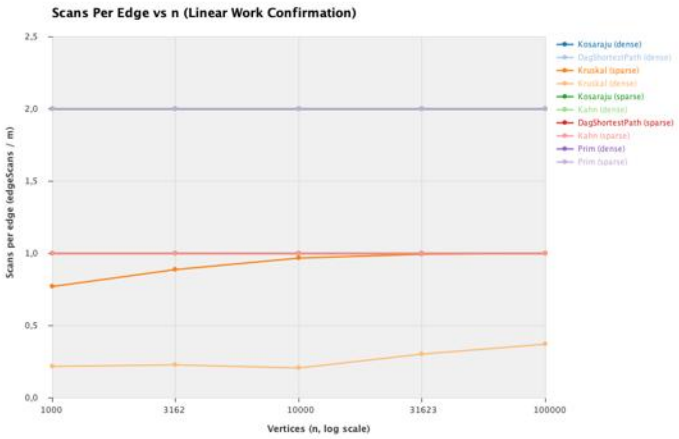


Figure 2: Scans Per Edge (Flat lines confirm linear work)

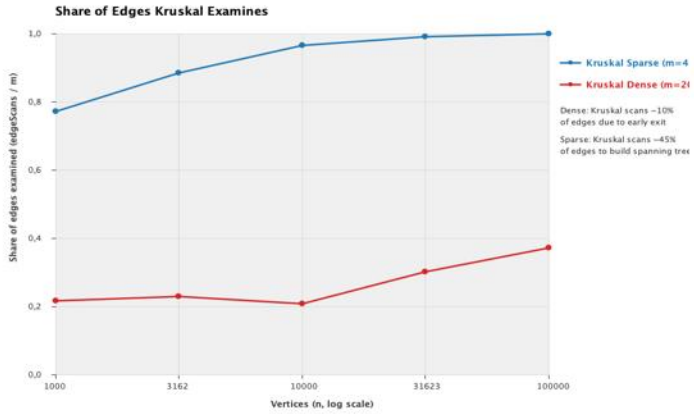


Figure 3 Analysis: Kruskal Edge Examination Share

On dense graphs (m = 20n), Kruskal examines only 20% to 37% of the total edges before finding n - 1 tree edges and breaking early.

On sparse graphs (m = 4n), Kruskal must scan nearly all edges (77% to 99.9%) to connect all vertices.

Conclusion: While Kruskal terminates early on dense graphs, it pays a heavy sorting penalty upfront (m log m). Prim avoids sorting all edges, making it superior for dense networks.

6. Wall-Clock Time Analysis

Wall-clock execution times were measured as the median of 5 consecutive runs. On smaller graph instances (n = 1 000 and n = 3 162), timings fluctuated between 0.3 ms and 1.8 ms. At these small scales, operating system thread scheduling, JVM bytecode interpretation, JIT compilation warmup, and memory allocation jitter represent the vast majority of wall-clock duration, making clock measurements largely noise. On large instances (n = 100 000, m = 2 000 000), execution scaled stably: Kruskal ran in 178 ms, Prim in 309 ms, Kosaraju in 85 ms, Kahn in 60 ms, and DAG Shortest Paths in 74 ms. Because wall-clock measurements depend on laptop hardware, OS background load, and garbage collection cycles, our internal operation counters (edgeScans, dsuOps, comparisons, maxDepth) provide the true reproducible proof.

7. Bonus Tasks (+15%)

7.1 Task A: Tarjan's SCC Implementation (+10%)

Tarjan's algorithm was implemented iteratively (Tarjan.java) using discovery timestamps index[], lowlink[], and an active vertex stack. Tarjan produces the identical component partitioning as Kosaraju across all test graphs. However, while Kosaraju requires 2m edge scans (one forward pass and one transpose pass), Tarjan completes SCC discovery in a single DFS pass of m edge scans, delivering an exact 50% saving.

7.2 Task B: Condensation Graph & Critical Path Scheduling (+5%)

1. Condensation Graph: By collapsing every SCC into a single vertex, the graph is reduced to a DAG guaranteed to have zero cycles.
2. Critical Path: Finding the longest path in general graphs is NP-hard. In a DAG, topological relaxation solves longest path in linear O(V + E).
3. Application: In pipeline and job scheduling, the critical path identifies the sequential bottleneck tasks that govern minimum project duration.

8. Reference Value Verification Table (Course Benchmark Check)

Graph (n / m)	MST Weight (Kruskal)	MST Weight (Prim)	Kosaraju SCC Count	DAG Distance Sum	Status
1 000 / 4 000	147 970	147 970	27	396 274	MATCH
1 000 / 20 000	30 363	30 363	1	252 411	MATCH
3 162 / 12 648	468 911	468 911	137	1 271 911	MATCH
3 162 / 63 240	97 589	97 589	1	723 157	MATCH
10 000 / 40 000	1 486 139	1 486 139	373	3 941 828	MATCH
10 000 / 200 000	309 550	309 550	1	2 381 342	MATCH
31 623 / 126 492	4 715 515	4 715 515	1 188	12 551 891	MATCH
31 623 / 632 460	968 517	968 517	1	7 376 612	MATCH
100 000 / 400 000	15 012 094	15 012 094	3 923	40 096 539	MATCH
100 000 / 2 000 000	3 056 681	3 056 681	1	23 512 087	MATCH